

HTTP Protocol

OrientDB RESTful HTTP protocol allows to talk with a [OrientDB Server instance](#) using the [HTTP protocol](#) and [JSON](#). OrientDB supports also a highly optimized [Binary protocol](#) for superior performances.

Available Commands

allocation DB's defragmentation	batch Batch of commands	class Operations on schema classes	cluster Operations on clusters
command Executes commands	connect Create the session	database Information about database	disconnect Disconnect session
document Operations on documents by RID GET - HEAD - POST - PUT - DELETE - PATCH	documentbyclass Operations on documents by Class	export Exports a database	function Executes a server-side function
index Operations on indexes	listDatabases Available databases	property Operations on schema properties	query Query
server Information about the server			

HTTP Methods

This protocol uses the four methods of the HTTP protocol:

- **GET**, to retrieve values from the database. It's idempotent that means no changes to the database happen. Remember that in IE6 the URL can be maximum of 2,083 characters. Other browsers supports longer URLs, but if you want to stay compatible with all limit to 2,083 characters

- **POST**, to insert values into the database
- **PUT**, to change values into the database
- **DELETE**, to delete values from the database

When using POST and PUT the following are important when preparing the contents of the post message:

- Always have the content type set to “application/json” or “application/xml”
- Where data or data structure is involved the content is in JSON format
- For OrientDB SQL or Gremlin the content itself is just text

Headers

All the requests must have these 2 headers:

```
'Accept-Encoding': 'gzip,deflate'  
'Content-Length': <content-length>
```

Where the `<content-length>` is the length of the request's body.

Syntax

The REST API is very flexible, with the following features:

- Data returned is in JSON format
- JSONP callback is supported
- Support for http and https connections
- The API itself is case insensitive
- API can just be used as a wrapper to retrieve (and control) data through requests written in OrientDB SQL or Gremlin
- You can avoid using `#` for RecordIDs in URLs, if you prefer. Just drop the `#` from the URL and it will still work

The REST syntax used is the same for all the four HTTP methods:

Syntax: `http://<server>:<port>/<command>/[<database>/<arguments>]`

Results are always in **JSON** format. Support for 'document' object types is through the use of the attribute `@type : 'd'`. This also applies when using inner document objects.

Example:

```
{
  "@type" : "d",
  "Name" : "Test",
  "Data" : { "@type": "d",
              "value": 0 },
  "@class" : "SimpleEntity"
}
```

JSONP is also supported by adding a *callback* parameter to the request (containing the callback function name).

Syntax: `http://<server>:<port>/<command>/[<database>/<arguments>]?callback=<callbackFunctionName>`

Commands are divided in two main categories:

- Server commands, such as to know server statistics and to create a new database
- Database commands, all the commands against a database

Authentication and security

All the commands (but the **Disconnect** need a valid authentication before to get executed. The OrientDB Server checks if the Authorization HTTP header is present, otherwise answers with a request of authentication (HTTP error code: 401).

The HTTP client (or the Internet Browser) must send user and password using the HTTP Base authentication. Password is encoded using Base64 algorithm. Please note that if you want to encrypt the password using a safe mode take in consideration to use SSL connections.

Server commands use the realm "OrientDB Server", while the database commands use a realm per database in this form: "OrientDB db-<database>", where <database> is the database name. In this way the Browser/HTTP client can reuse user and password inserted multiple times until the session expires or the "Disconnect" is called.

On first call (or when the session is expired and a new authentication is required), OrientDB returns the OSESSIONID parameter in response's HTTP header. On further calls the client should pass this OSESSIONID header in the requests and OrientDB will skip the authentication because a session is alive. By default sessions expire after 300 seconds (5 minutes), but you can change this configuration by setting the global setting:

```
network.http.sessionExpireTimeout
```

JSON data type handling and Schema-less mode

Since OrientDB supports also schema-less/hybrid modes how to manage types? JSON doesn't support all the types OrientDB has, so how can I pass the right type when it's not defined in the schema?

The answer is using the special field **"@fieldTypes"** as string containing all the field types separated by comma. Example:

```
{ "@class": "Account", "date": 1350426789, "amount": 100.34,
  "@fieldTypes": "date=t,amount=c" }
```

The supported special types are:

- 'f' for float
- 'c' for decimal
- 'l' for long
- 'd' for double
- 'b' for byte and binary
- 'a' for date
- 't' for datetime
- 's' for short
- 'e' for Set, because arrays and List are serialized as arrays like [3,4,5]
- 'x' for links
- 'n' for linksets
- 'z' for linklist
- 'm' for linkmap
- 'g' for linkbag
- 'u' for custom

Keep-Alive

Attention: OrientDB HTTP API utilizes [Keep-Alive](#) feature for better performance: the TCP/IP socket is kept open avoiding the creation of a new one for each command. If you need to re-authenticate, open a new connection avoiding to reuse the already open one. To force closing put "Connection: close" in the request header.

HTTP commands

Connect

GET - Connect

Connect to a remote server using basic authentication.

Syntax: `http://<server>:[<port>]/connect/<database>`

Example

HTTP GET request: `http://localhost:2480/connect/demo` HTTP response: 204 if ok, otherwise 401.

Database

GET - Database

HTTP GET request: `http://localhost:2480/database/demo` HTTP response:

```
{
  "server": {
    "version": "1.1.0-SNAPSHOT",
    "osName": "Windows 7",
    "osVersion": "6.1",
    "osArch": "amd64",
    "javaVendor": "Oracle Corporation",
    "javaVersion": "23.0-b21"
  }, "classes": [],
  ...
}
```

POST - Database

Create a new database.

Syntax: `http://<server>:[<port>]/database/<database>/<type>`

HTTP POST request: `http://localhost:2480/database/demo/plocal`

HTTP response: `{ "classes" : [], "clusters": [], "users": [], "roles": [], "config": [], "properties": {} }`

Class

GET - Class

Gets informations about requested class.

Syntax: `http://<server>[:<port>]/class/<database>/<class-name>`

HTTP response:

```
{ "class": {
  "name": "<class-name>"
  "properties": [
    { "name": <property-name>,
      "type": <property-type>,
      "mandatory": <mandatory>,
      "notNull": <not-null>,
      "min": <min>,
      "max": <max>
    }
  ]
}
```

For more information about properties look at the [supported types](#), or see the [SQL Create property](#) page for text values to be used when getting or posting class commands

Example

HTTP GET request: `http://localhost:2480/class/demo/0Function` HTTP response:

```
{
  "name": "OFunction",
  "superClass": "",
  "alias": null,
  "abstract": false,
  "strictmode": false,
  "clusters": [
    7
  ],
  "defaultCluster": 7,
  "records": 0,
  "properties": [
    {
      "name": "language",
      "type": "STRING",
      "mandatory": false,
      "readonly": false,
      "notNull": false,
      "min": null,
      "max": null,
      "collate": "default"
    },
    {
      "name": "name",
      "type": "STRING",
      "mandatory": false,
      "readonly": false,
      "notNull": false,
      "min": null,
      "max": null,
      "collate": "default"
    },
    {
      "name": "idempotent",
      "type": "BOOLEAN",
      "mandatory": false,
      "readonly": false,
      "notNull": false,
      "min": null,
      "max": null,
      "collate": "default"
    },
    {
      "name": "code",
      "type": "STRING",
      "mandatory": false,
      "readonly": false,
      "notNull": false,
      "min": null,
      "max": null,
      "collate": "default"
    },
    {
      "name": "parameters",
      "linkedType": "STRING",
      "type": "EMBEDDEDLIST",
```

```
    "mandatory": false,  
    "readonly": false,  
    "notNull": false,  
    "min": null,  
    "max": null,  
    "collate": "default"  
  }  
]  
}
```

POST - Class

Create a new class where the schema of the vertexes or edges is known. OrientDB allows (encourages) classes to be derived from other class definitions – this is achieved by using the **COMMAND** call with an OrientDB SQL command. Returns the id of the new class created.

Syntax: `http://<server>:[<port>]/class/<database>/<class-name>`

HTTP POST request: `http://localhost:2480/class/demo/Address2` HTTP response: `9`

Property

POST - Property

Create one or more properties into a given class. Returns the number of properties of the class.

Single property creation

Syntax: `http://<server>:[<port>]/property/<database>/<class-name>/<property-name>/[<property-type>]`

Creates a property named `<property-name>` in `<class-name>`. If `<property-type>` is not specified the property will be created as STRING.

Multiple property creation

Syntax: `http://<server>:[<port>]/property/<database>/<class-name>/`

Requires a JSON document post request content:

```
{
  "fieldName": {
    "propertyType": "<property-type>"
  },
  "fieldName": {
    "propertyType": "LINK",
    "linkedClass": "<linked-class>"
  },
  "fieldName": {
    "propertyType": "<LINKMAP|LINKLIST|LINKSET>",
    "linkedClass": "<linked-class>"
  },
  "fieldName": {
    "propertyType": "<LINKMAP|LINKLIST|LINKSET>",
    "linkedType": "<linked-type>"
  }
}
```

Example

Single property:

String Property Example: HTTP POST request: <http://localhost:2480/class/demo/simpleField> HTTP response: 1

Type Property Example: HTTP POST request: <http://localhost:2480/class/demo/dateField/DATE> HTTP response: 1

Link Property Example: HTTP POST request: <http://localhost:2480/class/demo/linkField/LINK/Person> HTTP response: 1

Multiple properties: HTTP POST request: <http://localhost:2480/class/demo/> HTTP POST content:

```
{
  "name": {
    "propertyType": "STRING"
  },
  "father": {
    "propertyType": "LINK",
    "linkedClass": "Person"
  },
  "addresses": {
    "propertyType": "LINKMAP",
    "linkedClass": "Address"
  },
  "examsRatings": {
    "propertyType": "LINKMAP",
    "linkedType": "INTEGER"
  }
  "events": {
    "propertyType": "LINKLIST",
    "linkedType": "DATE"
  }
  "family": {
    "propertyType": "LINKLIST",
    "linkedClass": "Person"
  }
  ...
}
```

HTTP response: 6

Cluster

GET - Cluster

Where the primary usage is a document db, or where the developer wants to optimise retrieval using the clustering of the database, use the CLUSTER command to browse the records of the requested cluster.

Syntax: `http://<server>:[<port>]/cluster/<database>/<cluster-name>/`

Where `<limit>` is optional and tells the maximum of records to load. Default is 20.

Example

HTTP GET request: `http://localhost:2480/cluster/demo/Address`

HTTP response:

```
{ "schema": {
  "id": 5,
  "name": "Address"
},
"result": [{
  "_id": "11:0",
  "_ver": 0,
  "@class": "Address",
  "type": "Residence",
  "street": "Piazza Navona, 1",
  "city": "12:0"
}]
...
}
```

Command

POST - Command

Execute a command against the database. Returns the records affected or the list of records for queries. Command executed via POST can be non-idempotent (look at [Query](#)).

Syntax: `http://<server>[:<port>]/command/<database>/<language>[/<command-text>[/limit[/<fetchPlan>]]]`

The content can be `<command-text>` or starting from v2.2 a json containing the command and parameters:

- by parameter name: `{"command":<command-text>, "parameters":{"<param-name>":<param-value>}}`
- by parameter position: `{"command":<command-text>, "parameters": [<param-value>]}`

Where:

- `<language>` is the name of the language between those supported. OrientDB distribution comes with "sql" and GraphDB distribution has both "sql" and "gremlin"
- `command-text` is the text containing the command to execute
- `limit` is the maximum number of record to return. Optional, default is 20
- `fetchPlan` is the fetching strategy to use. For more information look at [Fetching Strategies](#). Optional, default is *:1 (1 depth level only)

The *command-text* can appear in either the URL or the content of the POST transmission. Where the command-text is included in the URL, it must be encoded as per normal URL encoding. By default the result is returned in JSON. To have the result in CSV, pass

"Accept: text/csv" in HTTP Request.

Starting from v2.2, the HTTP payload can be a JSON with both command to execute and parameters. Example:

Execute a query passing parameters by name:

```
{
  "command": "select from V where name = :name and city = :city",
  "parameters": {
    "name": "Luca",
    "city": "Rome"
  }
}
```

Execute a query passing parameters by position:

```
{
  "command": "select from V where name = ? and city = ?",
  "parameters": [ "Luca", "Rome" ]
}
```

Read the [SQL section](#) or the [Gremlin introduction](#) for the type of commands.

Example

HTTP POST request: `http://localhost:2480/command/demo/sql` content: `update
Profile set online = false`

HTTP response: `10`

Or the same:

HTTP POST request: `http://localhost:2480/command/demo/sql/update Profile set
online = false`

HTTP response: `10`

Extract the user list in CSV format using curl

```
curl --user admin:admin --header "Accept: text/csv" -d "select from ouser"  
"http://localhost:2480/command/GratefulDeadConcerts/sql"
```

Batch

POST - Batch

Executes a batch of operations in a single call. This is useful to reduce network latency issuing multiple commands as multiple requests. Batch command supports transactions as well.

Syntax: `http://<server>:[<port>]/batch/<database>`

Content: `{ "transaction" : <true|false>, "operations" : [ { "type" : "" }* ] }`

Returns: the result of last operation.

Where: *type* can be:

- 'c' for create, 'record' field is expected.
- 'u' for update, 'record' field is expected.
- 'd' for delete. The '@rid' field only is needed.
- 'cmd' for commands (Since v1.6). The expected fields are:
 - 'language', between those supported (sql, gremlin, script, etc.)
 - 'command' as the text of the command to execute
- 'script' for scripts (Since v1.6). The expected fields are:
 - 'language', between the language installed in the JVM. Javascript is the default one, but you can also use SQL (see below)
 - 'script' as the text of the script to execute

Example

```
{ "transaction" : true,
  "operations" : [
    { "type" : "u",
      "record" : {
        "@rid" : "#14:122",
        "name" : "Luca",
        "vehicle" : "Car"
      }
    }, {
      "type" : "d",
      "record" : {
        "@rid" : "#14:100"
      }
    }, {
      "type" : "c",
      "record" : {
        "@class" : "City",
        "name" : "Venice"
      }
    }, {
      "type" : "cmd",
      "language" : "sql",
      "command" : "create edge Friend from #10:33 to #11:33"
    }, {
      "type" : "script",
      "language" : "javascript",
      "script" : "orient.getGraph().createVertex('class:Account')"
    }
  ]
}
```

SQL batch

Execute a POST against /batch URL by sending a payload in this format:

Syntax:

```
{ "transaction" : false,
  "operations" : [
    {
      "type" : "script",
      "language" : "sql",
      "script" : <text>
    }
  ]
}
```

Example:

```
{ "transaction" : false,
  "operations" : [
    {
      "type" : "script",
      "language" : "sql",
      "script" : [ "BEGIN;let account = CREATE VERTEX Account SET name =
'Luke';let city =SELECT FROM City WHERE name = 'London';CREATE EDGE Lives
FROM $account TO $city;COMMIT RETRY 100" ]
    }
  ]
}
```

To separate commands use semicolon (;). Starting from release 1.7 the “script” property can be an array of strings to put each command on separate item, example:

```
{ "transaction" : true,
  "operations" : [
    {
      "type" : "script",
      "language" : "sql",
      "script" : [ "LET account = CREATE VERTEX Account SET name = 'Luke'",
                  "LET city = SELECT FROM City WHERE name = 'London'",
                  "CREATE EDGE Lives FROM $account TO $city RETRY 100" ]
    }
  ]
}
```

Function

POST and GET - Function

Executes a server-side function against the database. Returns the result of the function that can be a string or a JSON containing the document(s) returned.

The difference between GET and POST method calls are if the function has been declared as idempotent. In this case can be called also by GET, otherwise only POST is accepted.

Syntax: `http://<server>[:<port>]/function/<database>/<name>[/<argument>*<server>`

Where

- `<name>` is the name of the function
- `<argument>`, optional, are the arguments to pass to the function. They are passed by position.

Creation of functions, when not using the Java API, can be done through the Studio in either Orient DB SQL or Java – see the [OrientDB Functions](#) page.

Example

HTTP POST request: `http://localhost:2480/function/demo/sum/3/5`

HTTP response: `8.0`

Database

GET - Database

Retrieve all the information about a database.

Syntax: `http://<server>[:<port>]/database/<database>`

Example

HTTP GET request: `http://localhost:2480/database/demo`

HTTP response:

```
{  
  "classes": [  
    {  
      "id": 0,  
      "name": "ORole",  
      "clusters": [3],  
      "defaultCluster": 3, "records": 0},  
    {  
      "id": 1,  
      "name": "OUser",  
      "clusters": [4],  
      "defaultCluster": 4, "records": 0},  
    {  
      ...  
    }  
  ]  
}
```

POST - Database

Create a new database. Requires additional authentication to the server.

Syntax for the url `http://:`

- *storage* can be
- 'plocal' for disk-based database
- 'memory' for in memory only database.
- *type*, is optional, and can be document or graph. By default is a document.

Example

HTTP POST request: `http://localhost:2480/database/demo2/local` HTTP response:

```
{ "classes": [  
  {  
    "id": 0,  
    "name": "ORole",  
    "clusters": [3],  
    "defaultCluster": 3, "records": 0},  
    {  
      "id": 1,  
      "name": "OUser",  
      "clusters": [4],  
      "defaultCluster": 4, "records": 0},  
    {  
      ...  
    }  
  ]  
}
```

DELETE - Database

Drop a database. Requires additional authentication to the server.

Syntax: `http://<server>[:<port>]/database/<databaseName>`

Where:

- **databaseName** is the name of database

Example

HTTP DELETE request: `http://localhost:2480/database/demo2` HTTP response code 204

Export

GET - Export

Exports a gzip file that contains the database JSON export.

Syntax: `http://[:]/export/`

HTTP GET request: `http://localhost:2480/export/demo2` HTTP response: demo2.zip file

Import

POST - Import

Imports a database from an uploaded JSON text file.

Syntax: `http://<server>[:<port>]/import/<database>`

The body of the HTTP call has to be the JSON of an exported DB (plain text). Multipart is not supported.

Important: Connect required: the connection with the selected database must be already established

Example

HTTP POST request: `http://localhost:2480/import/` HTTP response: returns a JSON object containing the result text *Success*:

```
{
  "responseText": "Database imported correctly"
}
```

_Fail::

```
{
  "responseText": "Error message"
}
```

List Databases

GET - List Databases

Retrieves the available databases.

Syntax: `http://<server>:<port>/listDatabases`

To let the Studio to display the database list by default the permission to list the database is assigned to guest. Remove this permission if you don't want anonymous user can display it.

For more details see [Server Resources](#)

Example of configuration of "guest" server user: a15b5e6bb7d06bd5d6c35db97e51400b

Example

HTTP GET request: `http://localhost:2480/listDatabases` HTTP response:

```
{
  "@type": "d", "@version": 0,
  "databases": ["demo", "temp"]
}
```

Disconnect

GET - Disconnect

Syntax: `http://<server>:[<port>]/disconnect`

Example

HTTP GET request: `http://localhost:2480/disconnect` HTTP response: empty.

Document

GET - Document

This is a key way to retrieve data from the database, especially when combined with a `<fetchplan>`. Where a single result is required then the RID can be used to retrieve that single document.

Syntax: `http://<server>:[<port>]/document/<database>/<record-id>[/<fetchPlan>]`

Where:

- `<record-id>` [See Concepts: RecordID](#)
- `<fetchPlan>` Optional, is the fetch plan used. 0 means the root record, -1 infinite depth, positive numbers is the depth level. Look at [Fetching Strategies](#) for more information.

Example

HTTP GET request: `http://localhost:2480/document/demo/9:0`

HTTP response can be:

- HTTP Code 200, with the document in JSON format in the payload, such as:

```
{
  "_id": "9:0",
  "_ver": 2,
  "@class": "Profile",
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "surname": "Garibaldi",
  "location": "11:0",
  "invitedBy": null,
  "sex": "male",
  "online": true
}
```

- HTTP Code 404, if the document was not found

The example above can be extended to return all the edges and vertices beneath #9:0

HTTP GET request: `http://localhost:2480/document/demo/9:0/*:-1`

HEAD - Document

Check if a document exists

Syntax: `http://<server>[:<port>]/document/<database>/<record-id>`

Where:

- `<record-id>` [See Concepts: RecordID](#)

Example

HTTP HEAD request: `http://localhost:2480/document/demo/9:0`

HTTP response can be:

- HTTP Code 204, if the document exists
- HTTP Code 404, if the document was not found

POST - Document

Create a new document. Returns the document with the new @rid assigned. Before 1.4.x the return was the @rid content only.

Syntax: `http://<server>[:<port>]/document/<database>`

Example

HTTP POST request: `http://localhost:2480/document/demo`

```
content:
{
  "@class": "Profile",
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "surname": "Garibaldi",
  "location": "11:0",
  "invitedBy": null,
  "sex": "male",
  "online": true
}
```

HTTP response, as the document created with the assigned [RecordID](#) as @rid:

```
{
  "@rid": "#11:4456",
  "@class": "Profile",
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "surname": "Garibaldi",
  "location": "11:0",
  "invitedBy": null,
  "sex": "male",
  "online": true
}
```

PUT - Document

Update a document. Remember to always pass the version to update. This prevent to update documents changed by other users (MVCC).

Syntax: `http://<server>[:<port>]/document/<database>[/<record-id>][?`

`updateMode=full|partial]` Where:

- **updateMode** can be **full** (default) or **partial**. With partial mode only the delta of changes is sent, otherwise the entire document is replaced (full mode)

Example

HTTP PUT request: `http://localhost:2480/document/demo/9:0`

```
content:
{
  "@class": "Profile",
  "@version": 3,
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "online": true
}
```

HTTP response, as the updated document with the updated @version field (Since v1.6):

```
content:
{
  "@class": "Profile",
  "@version": 4,
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "online": true
}
```

PATCH - Document

Update a document with only the difference to apply. Remember to always pass the version to update. This prevent to update documents changed by other users (MVCC).

Syntax: `http://<server>[:<port>]/document/<database>[/<record-id>]` Where:

Example

This is the document 9:0 before to apply the patch:

```
{
  "@class": "Profile",
  "@version": 4,
  "name": "Jay",
  "amount": 10000
}
```

HTTP PATCH request: `http://localhost:2480/document/demo/9:0`

```
content:
{
  "@class": "Profile",
  "@version": 4,
  "amount": 20000
}
```

HTTP response, as the updated document with the updated @version field (Since v1.6):

```
content:
{
  "@class": "Profile",
  "@version": 5,
  "name": "Jay",
  "amount": 20000
}
```

DELETE - Document

Delete a document.

Syntax: `http://<server>:[<port>]/document/<database>/<record-id>`

Example

HTTP DELETE request: `http://localhost:2480/document/demo/9:0`

HTTP response: empty

Document By Class

GET Document by Class

Retrieve a document by cluster name and record position.

Syntax: `http://<server>:[<port>]/documentbyclass/<database>/<class-name>/<record-position>[/fetchPlan]`

Where:

- `<class-name>` is the name of the document's class
- `<record-position>` is the absolute position of the record inside the class' default cluster
- `<fetchPlan>` Optional, is the fetch plan used. 0 means the root record, -1 infinite depth, positive numbers is the depth level. Look at [Fetching Strategies](#) for more information.

Example

HTTP GET request: `http://localhost:2480/documentbyclass/demo/Profile/0`

HTTP response:

```
{
  "_id": "9:0",
  "_ver": 2,
  "@class": "Profile",
  "nick": "GGaribaldi",
  "followings": [],
  "followers": [],
  "name": "Giuseppe",
  "surname": "Garibaldi",
  "location": "11:0",
  "invitedBy": null,
  "sex": "male",
  "online": true
}
```

HEAD - Document by Class

Check if a document exists

Syntax: `http://<server>:[<port>]/documentbyclass/<database>/<class-name>/<record-position>`

Where:

- `<class-name>` is the name of the document's class
- `<record-position>` is the absolute position of the record inside the class' default cluster

Example

HTTP HEAD request: `http://localhost:2480/documentbyclass/demo/Profile/0`

HTTP response can be:

- HTTP Code 204, if the document exists
- HTTP Code 404, if the document was not found

Allocation

GET - Allocation

Retrieve information about the storage space of a disk-based database.

Syntax: `http://<server>[:<port>]/allocation/<database>`

Example

HTTP GET request: `http://localhost:2480/allocation/demo`

HTTP response: `{ "size": 61910, "segments": [ {"type": "d", "offset": 0, "size": 33154}, {"type": "h", "offset": 33154, "size": 4859}, {"type": "h", "offset": 3420, "size": 9392}, {"type": "d", "offset": 12812, "size": 49098} ], "dataSize": 47659, "dataSizePercent": 76, "holesSize": 14251, "holesSizePercent": 24 }`

Index

NOTE: Every single new database has the default manual index called "dictionary".

GET - Index

Retrieve a record looking into the index.

Syntax: `http://<server>[:<port>]/index/<database>/<index-name>/<key>`

Example

HTTP GET request: `http://localhost:2480/index/demo/dictionary/test` HTTP response:

```
{
  "name" : "Jay",
  "surname" : "Miner"
}
```

PUT - Index

Create or modify an index entry.

Syntax: `http://<server>:[<port>]/index/<database>/<index-name>/<key>`

Example

HTTP PUT request: `http://localhost:2480/index/demo/dictionary/test` content: `{ "name" : "Jay", "surname" : "Miner" }`

HTTP response: No response.

DELETE - Index

Remove an index entry.

Syntax: `http://<server>:[<port>]/index/<database>/<index-name>/<key>`

Example

HTTP DELETE request: `http://localhost:2480/index/demo/dictionary/test` HTTP response: No response.

Query

GET - Query

Execute a query against the database. Query means only idempotent commands like SQL SELECT and TRAVERSE. Idempotent means the command is read-only and can't change the database. Remember that in IE6 the URL can be maximum of 2,083 characters. Other

browsers supports longer URLs, but if you want to stay compatible with all limit to 2,083 characters.

Syntax: `http://<server>[:<port>]/query/<database>/<language>/<query-text>[/<limit>][/<fetchPlan>]`

Where:

- `<language>` is the name of the language between those supported. OrientDB distribution comes with "sql" only. Gremlin language cannot be executed with **query** because it cannot guarantee to be idempotent. To execute Gremlin use [command](#) instead.
- `query-text` is the text containing the query to execute
- `limit` is the maximum number of record to return. Optional, default is 20
- `fetchPlan` is the fetching strategy to use. For more information look at [Fetching Strategies](#). Optional, default is *:1 (1 depth level only)

Other key points:

- To use commands that change the database (non-idempotent), see the [POST – Command section](#)
- The command-text included in the URL must be encoded as per a normal URL
- See the [SQL section](#) for the type of queries that can be sent

Example

HTTP GET request: `http://localhost:2480/query/demo/sql/select from Profile`

HTTP response:

```
{ "result": [
  {
    "_id": "-3:1",
    "_ver": 0,
    "@class": "Address",
    "type": "Residence",
    "street": "Piazza di Spagna",
    "city": "-4:0"
  },
  {
    "_id": "-3:2",
    "_ver": 0,
    "@class": "Address",
    "type": "Residence",
    "street": "test",
    "city": "-4:1"
  }
] }
```

The same query with the limit to maximum 20 results using the fetch plan `*:-1` that means load all recursively:

HTTP GET request: `http://localhost:2480/query/demo/sql/select from Profile/20/*:-1`

Server

GET - Server

Retrieve information about the connected OrientDB Server. Requires additional authentication to the server.

Syntax: `http://<server>[:<port>]/server`

Example

HTTP GET request: `http://localhost:2480/server` HTTP response:

```
{
  "connections": [{
    "id": "4",
    "id": "4",
    "remoteAddress": "0:0:0:0:0:0:0:1:52504",
    "db": "-",
    "user": "-",
    "protocol": "HTTP-DB",
    "totalRequests": "1",
    "commandInfo": "Server status",
    "commandDetail": "-",
    "lastCommandOn": "2010-05-26 05:08:58",
    "lastCommandInfo": "-",
    "lastCommandDetail": "-",
    "lastExecutionTime": "0",
    "totalWorkingTime": "0",
    ...
  ]
}
```

POST - Server

Changes server configuration. Supported configuration are:

- any setting contained in `OGlobalConfiguration` class, by using the prefix `configuration` in setting-name
- logging level, by using the prefix `log` in setting-name

Syntax: `http://<server>:[<port>]/server/<setting-name>/<setting-value>`

Example

Example on changing the server log level to FINEST

```
localhost:2480/server/log.console/FINEST
```

Example on changing the default timeout for query to 10 seconds

```
localhost:2480/server/configuration.command.timeout/10000
```

Connection

POST - Connection

Syntax: `http://<server>:[<port>]/connection/<command>/<id>`

Where:

- **command** can be:
- **kill** to kill a connection
- **interrupt** to interrupt the operation (if possible)
- **id**, as the connection id. To know all the connections use GET /connection/[<db>]

You've to execute this command authenticated in the OrientDB Server realm (no database realm), so get the root password from config/orientdb-server-config.xml file (last section).